

Character Devices und Treiberstrukturen

Gerätenummer, cdev, file_operations und Lifecycle zu einem belastbaren Treiber verbinden

3.10

Character Devices liefern sequenzielle I/O-Semantik

01 Dateiartig

Anwendungen nutzen open, read, write, poll und ioctl über einen Dateideskriptor.

02 Treibergeführt

Der Treiber entscheidet über Datenquelle, Blockierung, Position und Fehlerbild.

03 Nicht blockorientiert

Keine Sektor- oder Page-Cache-Semantik wie bei Blockgeräten.

04 Instanzbezogen

Minor-Nummer und private_data führen zur konkreten Geräteinstanz.

Typische Beispiele

Sensorstrom · Steuerkanal · FPGA-Endpunkt · einfache virtuelle Geräte

Vier Objekte bilden den Zugriffspfad



`open()` löst den Knoten auf; VFS verbindet die Geräteerkennung mit dem `cdev` und dessen Operationen.

Der Pfadname ist nicht die Identität des Treibers – die registrierte Gerätenummer ist es.

Major und Minor teilen Familie und Instanz

```
dev_t dev = MKDEV(major, minor);  
  
unsigned int major = MAJOR(dev);  
unsigned int minor = MINOR(dev);
```

MAJOR Treiber- oder Gerätefamilie

MINOR konkrete Instanz oder Kanal

Beispiel

/dev/demo0 → 240:0 /dev/demo1 → 240:1

Major-Werte nicht fest einbauen: dynamische Vergabe vermeidet Konflikte.

Gerätenummern werden als Bereich reserviert

```
dev_t base;  
int ret;  
  
ret = alloc_chrdev_region(&base, 0, count, "demo");  
if (ret)  
    return ret;  
  
pr_info("major=%u\n", MAJOR(base));
```

01

Eingabe

baseminor · count · Name

02

Ausgabe

erste dev_t in base

03

Erfolg

0

04

Fehler

negativer errno

Symmetrischer Rückbau: unregister_chrdev_region(base, count)

struct cdev bindet Nummern an Operationen

```
struct demo_dev {  
    struct cdev cdev;  
    struct mutex lock;  
    char data[128];  
    size_t len;  
};
```

1 Initialisieren

```
cdev_init(&d->cdev, &demo_fops);
```

2 Veröffentlichen

```
ret = cdev_add(&d->cdev, devt, 1);
```

cdev_add() macht den Endpunkt sofort erreichbar: Zustand und Synchronisation müssen vorher bereit sein.

file_operations definiert die VFS Integration

```
static const struct file_operations demo_fops = {  
    .owner    = THIS_MODULE,  
    .open     = demo_open,  
    .read     = demo_read,  
    .write    = demo_write,  
    .release  = demo_release,  
    .llseek   = no_llseek,  
};
```

open

Instanz vorbereiten

read / write

Daten übertragen

release

Open-Zustand lösen

llseek

Positionssemantik festlegen

Nicht implementierte Operationen bleiben NULL; nur bewusst unterstützte Semantik anbieten.

Gerätezustand gehört in eine eigene Struktur

```
struct demo_dev
```

- cdev VFS-Anbindung
- lock Nebenläufigkeit
- buffer / FIFO Datenzustand
- device * Device Model
- HW-Ressourcen IRQ · MMIO · Bus

Prinzip

Ein Objekt pro Geräteinstanz

Callbacks erhalten keine globalen Annahmen, sondern lösen die passende Instanz auf.

- mehrere Minor-Nummern
- getrennte Locks und Buffer
- klarer Lifecycle

Globale Variablen skalieren schlecht und verschleiern Besitzverhältnisse.

open verbindet inode, cdev und private_data

```
static int demo_open(struct inode *inode,
                    struct file *file)
{
    struct demo_dev *d;

    d = container_of(inode->i_cdev,
                    struct demo_dev, cdev);
    file->private_data = d;
    return 0;
}
```

- 01 `inode->i_cdev`
registrierten Endpunkt finden
- 02 `container_of()`
umschließende Instanz gewinnen
- 03 `file->private_data`
Folgezugriffe beschleunigen

read, write und release greifen danach direkt auf `file->private_data` zu.

read und write arbeiten auf der Geräteinstanz

READ

```
struct demo_dev *d = file->private_data;

mutex_lock(&d->lock);
/* Snapshot bilden */
mutex_unlock(&d->lock);

return simple_read_from_buffer(...);
```

WRITE

```
struct demo_dev *d = file->private_data;

/* Userdaten kopieren und prüfen */
mutex_lock(&d->lock);
/* Zustand atomar aktualisieren */
mutex_unlock(&d->lock);

return count;
```

Lock-Grenzen schützen Kernelzustand – Usercopy und Schlafverhalten müssen bewusst eingeplant werden.

Device Model macht das Gerät auffindbar

```
demo_class = class_create("demo");  
if (IS_ERR(demo_class))  
    return PTR_ERR(demo_class);  
  
dev = device_create(demo_class, NULL, devt, d,  
                  "demo%d", minor);  
if (IS_ERR(dev))  
    return PTR_ERR(dev);
```

1

class_create

Klasse unter /sys/class/demo

2

device_create

Geräteobjekt + dev-Attribut

3

uevent

Userspace wird benachrichtigt

4

udev

erzeugt typischerweise /dev/demo0

device_create() registriert im Device Model; den /dev-Knoten erzeugt meist der Userspace-Gerätemanager.

Initialisierung folgt der Sichtbarkeitsgrenze

- | | | |
|---|------------------------------------|--------------------------|
| 1 | Instanz allokieren | noch unsichtbar |
| 2 | Locks und Zustand initialisieren | noch unsichtbar |
| 3 | dev_t reservieren | Nummer belegt |
| 4 | cdev initialisieren und hinzufügen | open möglich |
| 5 | Klasse und Device registrieren | Userspace entdeckt Gerät |

Ab `cdev_add()` können Callbacks laufen – veröffentlichen ist kein unverbindlicher Zwischenschritt.

Fehlerpfade rollen in umgekehrter Reihenfolge zurück

```
ret = alloc_chrdev_region(&devt, 0, 1, "demo");
if (ret)
    return ret;

cdev_init(&d->cdev, &demo_fops);
ret = cdev_add(&d->cdev, devt, 1);
if (ret)
    goto err_region;

demo_class = class_create("demo");
if (IS_ERR(demo_class)) {
    ret = PTR_ERR(demo_class);
    goto err_cdev;
}
/* ... */
err_cdev:    cdev_del(&d->cdev);
err_region: unregister_chrdev_region(devt, 1);
return ret;
```

Rückbau-Regel

device_create	←	device_destroy
class_create	←	class_destroy
cdev_add	←	cdev_del
alloc_chrdev_region	←	unregister_chrdev_region

Nur erfolgreich erworbene Ressourcen freigeben; den ursprünglichen Fehlercode erhalten.

Entfernen stoppt neue Opens, nicht bestehende fds

**Wichtig**

Nach `cdev_del()` können bereits geöffnete Dateideskriptoren ihre Callbacks weiterhin aufrufen.

A**Lifecycle**

Instanz erst nach letztem Nutzer freigeben.

B**Abwesenheit**

Callbacks bei entferntem Gerät mit `-ENODEV` beenden.

Fünf Regeln ergeben einen belastbaren Char-Treiber

01 Erst initialisieren

Alle Callback-Daten vor `cdev_add` bereitstellen.

02 Instanzen kapseln

`cdev`, Locks, Buffer und Device gemeinsam besitzen.

03 Dynamisch nummerieren

Bereiche reservieren und Minorwerte bewusst planen.

04 Symmetrisch abbauen

Jeden erfolgreichen Schritt exakt rückgängig machen.

05 Offene fds respektieren

Entfernung und Objekt-Lifecycle getrennt behandeln.

NÄCHSTER SCHRITT

Praxis: Einen einfachen Character-Device-Treiber implementieren und testen